

instruction	usage (asm)	operation (rtl logic)	note (中文提示)
alu: 算术逻辑			
add.w	add.w rd, rj, rk	$rd = rj + rk$	32位加法 (忽略溢出)
sub.w	sub.w rd, rj, rk	$rd = rj - rk$	32位减法
slt	slt rd, rj, rk	$rd = (\$signed(rj) < \$signed(rk))$	有符号比较, 结果置0或1
sltu	sltu rd, rj, rk	$rd = (\$unsigned(rj) < \$unsigned(rk))$	无符号比较
nor	nor rd, rj, rk	$\text{`}rd = \sim(rj$	$rk)\text{'}$
and	and rd, rj, rk	$rd = rj \& rk$	按位与
or	or rd, rj, rk	$\text{`}rd = rj$	$rk\text{'}$
xor	xor rd, rj, rk	$rd = rj \wedge rk$	按位异或
alu: 立即数			
addi.w	addi.w rd, rj, imm	$rd = rj + sext(imm12)$	imm 符号扩展
slti	slti rd, rj, imm	$rd = (\$signed(rj) < sext(imm12))$	imm 符号扩展, 有符号比
sltui	sltui rd, rj, imm	$rd = (\$unsigned(rj) < sext(imm12))$	imm 符号扩展, 无符号比 (注意!)
andi	andi rd, rj, imm	$rd = rj \& zext(imm12)$	imm 零扩展 (高位补0)
ori	ori rd, rj, imm	$\text{`}rd = rj$	$zext(imm12)\text{'}$
xori	xori rd, rj, imm	$rd = rj \wedge zext(imm12)$	imm 零扩展
lu12i.w	lu12i.w rd, imm	$rd = \{imm20, 12'b0\}$	加载立即数到高20位
shift: 移位			
sll.w	sll.w rd, rj, rk	$rd = rj \ll rk[4:0]$	逻辑左移
srl.w	srl.w rd, rj, rk	$rd = rj \gg rk[4:0]$	逻辑右移 (补0)
sra.w	sra.w rd, rj, rk	$rd = \$signed(rj) \ggg rk[4:0]$	算术右移 (补符号位)
slli.w	slli.w rd, rj, imm	$rd = rj \ll imm[4:0]$	立即数逻辑左移
srli.w	srli.w rd, rj, imm	$rd = rj \gg imm[4:0]$	立即数逻辑右移
srai.w	srai.w rd, rj, imm	$rd = \$signed(rj) \ggg imm[4:0]$	立即数算术右移
mul/div: 乘除			
mul.w	mul.w rd, rj, rk	$rd = (rj * rk)[31:0]$	乘法取低32位
mulh.w	mulh.w rd, rj, rk	$rd = (\$signed(rj) * \$signed(rk))[63:32]$	有符号乘, 取高32位
mulh.wu	mulh.wu rd, rj, rk	$rd = (\$unsigned(rj) * \$unsigned(rk))[63:32]$	无符号乘, 取高32位
div.w	div.w rd, rj, rk	$rd = \$signed(rj) / \$signed(rk)$	有符号除

mod.w	mod.w rd, rj, rk	rd = \$signed(rj) % \$signed(rk)	有符号求余
div.wu	div.wu rd, rj, rk	rd = \$unsigned(rj) / \$unsigned(rk)	无符号除
mod.wu	mod.wu rd, rj, rk	rd = \$unsigned(rj) % \$unsigned(rk)	无符号求余
memory: 访存			
ld.w	ld.w rd, rj, imm	vaddr=rj+sext(imm12); rd=mem[vaddr]	读32位字
st.w	st.w rd, rj, imm	vaddr=rj+sext(imm12); mem[vaddr]=rd	写32位字
ld.b	ld.b rd, rj, imm	rd = sext(mem[vaddr][7:0])	读字节 -> 符号扩展
ld.bu	ld.bu rd, rj, imm	rd = zext(mem[vaddr][7:0])	读字节 -> 零扩展
ld.h	ld.h rd, rj, imm	rd = sext(mem[vaddr][15:0])	读半字 -> 符号扩展
ld.hu	ld.hu rd, rj, imm	rd = zext(mem[vaddr][15:0])	读半字 -> 零扩展
st.b	st.b rd, rj, imm	mem[vaddr] = rd[7:0]	写 rd 的低8位
st.h	st.h rd, rj, imm	mem[vaddr] = rd[15:0]	写 rd 的低16位
branch: 跳转			
beq	beq rj, rd, offs	if(rj==rd) pc += sext(offs16)<<2	相等跳 (相对寻址)
bne	bne rj, rd, offs	if(rj!=rd) pc += sext(offs16)<<2	不等跳 (相对寻址)
b	b offs	pc += sext(offs26)<<2	无条件跳 (相对寻址)
bl	bl offs	r1=pc+4; pc += sext(offs26)<<2	函数调用 (链接到 \$r1)
jirl	jirl rd, rj, offs	tmp=pc+4; pc=rj+sext(offs16)<<2; rd=tmp	间接跳/返回 (基址+偏移)
system: 系统			
csrrd	csrrd rd, csr_num	rd = csr[csr_num]	读 csr
csrwr	csrwr rd, csr_num	old=csr; csr=rd; rd=old	写 csr, 返回旧值
csrxchg	csrxchg rd, rj, id	`mask=rj; old=csr; csr=(old&~mask)	(rd&mask)`
ertn	ertn	pc = era; status = prmd	异常返回 (必须实现)
syscall	syscall code	raise_exception(0xb)	系统调用异常
break	break code	raise_exception(0xc)	断点异常